

Improving policy training for autonomous driving through randomized ensembled double Q-learning with Transformer encoder feature evaluation

Jie Fan, Xudong Zhang^{*}, Yuan Zou, Yuanyuan Li, Yingqun Liu, Wenjing Sun

National Engineering Research Center for Electric Vehicles, School of Mechanical Engineering, Beijing Institute of Technology, No. 5 Zhongguancun South Street, Haidian District, 100081, Beijing, China

ARTICLE INFO

Keywords:

Autonomous driving
Reinforcement learning
Transformer encoder

ABSTRACT

In the burgeoning field of autonomous driving, reinforcement learning (RL) has gained prominence for its adaptability and intelligent decision-making. However, conventional RL methods face challenges in efficiently extracting relevant features from high-dimensional inputs and maximizing the use of environment-agent interaction data. To surmount these obstacles, this paper introduces a novel RL-based approach that integrates randomized ensembled double Q-Learning (REDQ) with a Transformer encoder. The Transformer encoder's attention mechanism is utilized to dynamically evaluate features according to their relevance in different driving scenarios. Simultaneously, the implementation of REDQ, characterized by a high update-to-data (UTD) ratio, enhances the utilization of interaction data during policy training. Especially, the incorporation of ensemble strategy and in-target minimization in REDQ significantly improves training stability, especially under high UTD conditions. Ablation studies indicate that the Transformer encoder exhibits significantly enhanced feature extraction capabilities compared to conventional network architectures, resulting in a 13.6% to 21.4% increase in success rate for the MetaDrive autonomous driving task. Additionally, when compared to standard RL methodologies, the proposed approach demonstrates a faster rate of reward acquisition and achieves a 67.5% to 69% improvement in success rate.

1. Introduction

The development of autonomous driving technology holds paramount importance in shaping the future of transportation [1,2]. It has the potential to revolutionize road safety by significantly reducing human error and enhance accessibility for individuals with limited mobility, providing newfound independence [3,4]. Reinforcement learning (RL)-based control methods have emerged as a transformative force in the realm of autonomous driving research, showcasing remarkable success [5,6]. By leveraging the principles of trial and error, these approaches enable vehicles to learn optimal decision-making strategies in complex and dynamic environments. RL algorithms empower autonomous systems to adapt and refine their behaviors over time through interactions with the surrounding environment, mimicking the learning processes observed in biological organisms [7]. This adaptability is particularly crucial in addressing the unpredictable nature of real-world driving scenarios, where traditional rule-based systems may fall short [8,9].

Despite the successful applications, current RL-based methods in autonomous driving still face two notable challenges. Firstly, the input

data for the autonomous driving system is typically rich and multifaceted, containing a plethora of information [10]. RL faces the critical task of evaluating meaningful features from high-dimensional inputs. Emphasizing the importance of specific input dimensions in different scenarios is crucial for refining the final policy and optimizing decision-making. Secondly, there is a pressing need to explore methods for enhancing the utilization of data generated through the interaction between the vehicle and its environment. Efficiently harnessing this interaction data can greatly expedite the training process, allowing autonomous systems to learn and adapt more swiftly.

To address the above two challenges, this paper proposes to improve policy training for autonomous driving through randomized ensembled double Q-learning (REDQ) with Transformer encoder feature evaluation. The main contributions of this paper are shown in Fig. 1 and described as follows:

- Firstly, the Transformer encoder block, which incorporates the self-attention module, is introduced to capture the internal relationship between the input observation domain and the output action domain in the context of autonomous driving. By assigning appropriate attention

^{*} Corresponding author.

E-mail addresses: 3120225209@bit.edu.cn (J. Fan), xudong.zhang@bit.edu.cn (X. Zhang), zouyuan@bit.edu.cn (Y. Zou), lyapplee@icloud.com (Y. Li), 18600522953@163.com (Y. Liu), sunw-enjing@163.com (W. Sun).

<https://doi.org/10.1016/j.asoc.2024.112386>

Received 30 January 2024; Received in revised form 1 September 2024; Accepted 14 October 2024

Available online 29 October 2024

1568-4946/© 2024 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

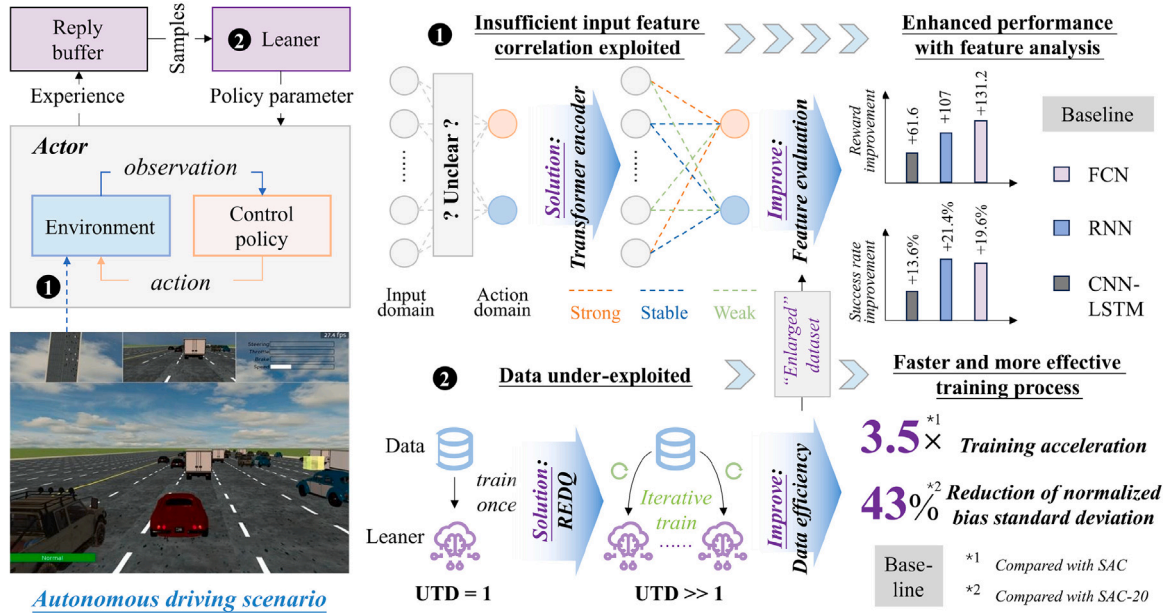


Fig. 1. Challenges in existing RL methods for autonomous driving and our solution.

weights to different observations in the input, the self-attention module could interpret complicated relationships among various features, allowing the model to understand the input–output dependencies.

- Secondly, REDQ is introduced to exploit the data efficacy to its fullest potential. Different from traditional model-free RL-based algorithms whose update-to-data (UTD) ratio (denoting the number of updates performed by the agent relative to the actual interactions with the environment) is usually equal to 1, REDQ could reach 20–40 through iterative training of multiple learners, thus greatly improving the sample efficiency.

- Moreover, while the Transformer encoder is powerful in capturing feature correlations and extracting relevant information from high-dimensional inputs, it is also known to be data-hungry. REDQ, with its high UTD ratio, complements the Transformer by effectively increasing the amount of training data. Through iteratively training multiple Q-learners on the same interaction data, REDQ simulates the effect of an expanded dataset. This “enlarged” dataset provides the Transformer with more opportunities to learn, leading to better feature evaluation and more accurate decision-making.

In summary, the Transformer encoder improves feature extraction, while REDQ ensures sufficient data utilization under the Transformer architecture, thereby creating a more efficient and accurate learning pipeline for autonomous driving. This integration allows the system to overcome the limitations of both components individually, yielding a robust solution that accelerates learning and improves overall performance.

The proposed method is validated in MetaDrive simulation environment [11]. Results indicate that the Transformer encoder could effectively assign appropriate weights to diverse inputs, adapting specifically to various driving scenarios. When compared to commonly utilized network structures, such as fully connected network (FCN), recurrent neural network (RNN), and convolutional neural network (CNN) combined with long-short term memory (LSTM) network, the proposed method shows a 61.6 to 131.2 improvement in reward and 13.6% to 21.4% increase in success rate. Additionally, when benchmarked against the soft actor-critic (SAC) and ensemble averaging (AVG) algorithms, the proposed method exhibits significantly reduced bias variation, highlighting its enhanced reliability and efficiency in decision-making processes for autonomous driving. Furthermore, in comparison to widely used RL algorithms such as twin delayed deep deterministic policy gradient (TD3) and trust region policy optimization

(TRPO), the proposed approach demonstrates an accelerated training process and achieves a 67.5% to 69% improvement in success rate.

The following paper will be organized as follows. Section 2 will give a description of existing RL methods in solving autonomous driving control problems. Section 3 will describe the general framework of the proposed method. Section 4 will validate the method in the MetaDrive environment and Section 5 concludes the whole paper.

2. Related works

RL-based autonomous driving control methods surpass traditional approaches, like rule-based methods, due to their adaptability and capacity for learning in dynamic environments [12]. Existing RL-based methods can generally be divided into two groups, namely model-based methods and model-free methods [13]. Model-based RL involves constructing an internal model of the environment to simulate possible outcomes, enabling agents to plan and optimize actions based on the acquired model [14]. The advantage of model-based RL lies in its ability to leverage a learned internal model of the environment, enabling more sample-efficient and effective decision-making by simulating potential outcomes and planning optimal actions [15]. Many existing research tried to apply model-based RL for autonomous driving. For example, Wu, J. et al. proposed an uncertainty-aware model-based RL method based on the adaptive truncation approach, which established an action-conditioned ensemble model with the capability of uncertainty assessment as the environment model [16]. Guan, Y. et al. addressed the complexity of intersection scenarios in autonomous driving by introducing a novel approach that formulates decision-making as an optimal path selection and tracking process, employing a model-based RL algorithm called generalized exterior point to efficiently find a neural network solution offline [17]. Mirchevska, B. et al. presented an amortized Q-learning with model-based action proposals integrated with a trajectory planner that successfully learns and executes optimal long-term decision-making strategies on highways for autonomous driving [18]. However, the efficacy of model-based RL methods is intricately tied to the accuracy of the constructed model. In the context of autonomous driving, characterized by high dynamism, creating a model that accurately encapsulates all possible driving scenarios is practically impossible. Consequently, the performance of model-based RL approaches in autonomous driving is constrained to a degree, struggling in situations where the model’s fidelity falls short.

In contrast to model-based RL methods, model-free approaches dispense with the need to construct an explicit model of the environment [19,20]. Model-free methods learn directly from collected data, relying on the acquired experiences to shape decision-making processes. This characteristic imparts model-free methods with notable advantages over their model-based counterparts [21]. The absence of a predefined environment model makes model-free techniques particularly well-suited for handling the inherent complexity and dynamism of scenarios like autonomous driving, where constructing an accurate model is challenging. For example, Toromanoff, M. et al. introduced a groundbreaking approach termed implicit affordances, harnessing model-free RL for the intricate task of urban driving, encompassing lane keeping, pedestrians and vehicles avoidance, and traffic light detection, with demonstrated success in complex driving scenarios [22]. Zhang, R. et al. presented ResRace, an efficient residual policy learning method for high-speed autonomous driving, combining a modified artificial potential field controller for navigation with a deep model-free RL algorithm to generate a residual policy [23]. Schier, M. et al. introduced an innovative model-free RL framework for automated driving, utilizing a heterogeneous graph to intuitively model the entire road network and traffic participants [24]. Li, J. proposed a hierarchical behavior planning framework using a high-level model-free RL agent as a coordinator for the low-level controllers [25]. In model-free RL agents, the feature evaluation component of the agent's network assumes a pivotal role in determining the efficacy of the final policy [26,27]. This is particularly crucial as the network is tasked with establishing the intricate relationship between observations, which typically comprise direct inputs from low-level sensors like cameras, Lidar, and inertial measurement units, as well as high-level scene information encompassing road details and nearby vehicles' information such as velocity and heading. The feature evaluation module in these studies incorporates various neural network architectures, including CNN [28], FCN [29], RNN [30], or hybrid combinations [31]. For instance, Wu, J. et al. proposed a real-time human-guidance-based deep RL method for policy training in an end-to-end autonomous driving scenario, utilizing an improved actor-critic architecture with FCNs to extract valuable features from vehicle states [32]. Szöke, L. et al. employed CNNs with LSTM layers as a function approximator for an autonomous driving agent to execute acceleration and lane changing [33]. Li, L. et al. used RNNs as feature extractors to infer the lane change intentions of autonomous vehicles on highways [34]. However, the inherent high-dimensionality of input features in autonomous driving poses a challenge. Commonly used network structures may struggle to discern the relative importance of different input features in shaping the final optimal action, potentially diminishing the overall performance of the agent's decision-making capabilities.

Recently, self-attention blocks [35], derived from the Transformer network [36], have gained significant traction owing to their capacity to capture global dependencies and relationships among all features. Notably, these blocks offer interpretability by indicating the significance of different positions in the input for each output position. Leveraging these advantages, self-attention blocks have witnessed notable success in diverse control domains [37,38]. For instance, Adam, A. B. et al. introduced an end-to-end multi-modal convolutional self-attention network for power control in non-orthogonal multiple access networks, employing self-attention blocks on a ResNet to extract high-quality features [39]. Liu, K. et al. innovatively applied the self-attention mechanism in the critic network of a multi-agent actor-critic structure to enhance learning efficiency in scenarios involving an increasing number of agents [40]. Wong, A. et al. proposed Fast GraspNet, a swift self-attention neural network architecture tailored for embedded multi-task learning in computer vision tasks related to robotic grasping [41]. These successes in control domains have inspired the exploration of self-attention blocks, especially Transformer that successfully applies self-attention into fields of computer vision and natural language process, in the context of autonomous driving. For

instance, Shao, H. et al. proposed a safety-enhanced autonomous driving framework called interpretable sensor fusion Transformer, designed to fully process and fuse information from multi-modal, multi-view sensors to achieve comprehensive scene understanding and adversarial event detection [42]. Sharma, O. et al. introduced a novel Transformer-based composite network to predict both driver behavior and the future trajectory of a target vehicle in highway autonomous driving scenarios [43]. Zhan, G. et al. utilized a Transformer encoder as the transformation module to extract features for state representation in autonomous driving systems [44]. Most current studies applying Transformer to autonomous driving primarily focus on environment perception and state representation. However, frameworks that integrate Transformer into end-to-end control systems for autonomous driving are still rare. An example of such research is by Ge, L. et al. who introduced deep RL navigation via decision Transformer to enhance the decision-making capabilities of autonomous vehicles, effectively modeling the temporal dependencies in observations and actions [45]. Nevertheless, this research does not fully explain the intertwined relationships between different state inputs, thereby not fully exploiting the attention mechanism inherent in the Transformer block.

Among all model-free RL methods, deep Q-learning methods dominate autonomous driving research due to their robust ability to handle high-dimensional state spaces and complex decision-making processes, enabling effective real-time performance and adaptability. For instance, Lu, J. et al. utilized parallel control to explore event-triggered deep RL, developing an event-triggered deep Q-network for autonomous driving decision-making without training an explicit triggering condition [46]. Mazouchi, M. et al. modeled the multilane road ahead as a finite-state nonstationary Markov decision process, solving the navigation problem with a risk-averse, preview-based Q-learning planner [47]. Nguyen, H. et al. developed a safe RL-based driving policy for highways, enhancing vehicle safety with a Q-learning table that stores the values corresponding to each action state [48]. Peng X. et al. proposed a double broad Q-network algorithm, replacing deep Q-network with the broad learning system to improve response time and safety in autonomous driving during lane-changing and overtaking maneuvers [49]. Guo J. et al. proposed a deep Q-network with a request-respond mechanism for optimizing simultaneous lane-changing maneuvers in congested traffic by dividing vehicles into isolated groups and training them in a decentralized manner, achieving comparable efficiency to game theory-based methods while significantly reducing computational time [50]. However, in these deep Q-learning-based model-free RL algorithms, the UTD ratio is one, meaning the agent performs only one update per actual interaction with the environment. This characteristic necessitates prolonged interactions between the agent and the environment, resulting in a slower convergence rate during training. Higher UTD ratios indicate increased efficiency in utilizing interaction-collected data for model improvement. Consequently, the limited UTD ratio represents a significant constraint that affects the performance of model-free RL models [51].

To address this challenge, Chen, X. proposed REDQ, which introduces multiple updates over a single agent-environment interaction, achieving a UTD ratio of 20–40 [52]. Additionally, it incorporates an ensemble of Q-functions and in-target minimization across a random subset of Q-functions, enhancing algorithm stability. REDQ surpasses traditional deep Q-learning by reducing overestimation bias and improving policy performance through the use of multiple Q-value estimates and enhanced training strategies. Successfully applied in robotics, REQD demonstrates rapid convergence, as evidenced by achieving quadruped locomotion learning in just 20 min in the real world [53]. Given the inherent challenge of self-attention blocks requiring a substantial amount of data to achieve convergence [54], particularly in the context of high-dimensional input features within autonomous driving scenarios, it may be more appropriate to apply the REDQ algorithm, thus not only addressing the data-intensive nature of training self-attention blocks but also aligning with the demand for swift feature evaluation, thereby enhancing the overall performance of autonomous driving systems.

3. Problem definition and methodology

In this section, we will first define the problem of policy training within the domain of autonomous driving and identify the associated challenges encountered in real-world scenarios. Next, we will present the overarching framework of our proposed methodology. Finally, we will introduce the training environments utilized, namely MetaDrive, and provide a fundamental overview of the key components integral to our approach.

3.1. Problem description

Policy training is a crucial aspect of RL for autonomous driving, focusing on determining an optimal mapping from state to action. In this context, state encompasses the current status of the vehicle along with surrounding traffic information, while action refers to the vehicle's controllable operations, including longitudinal control (accelerating/braking) and lateral control (steering). The goal of policy training is to identify an optimal state-to-action mapping that increases the cumulative reward, so as to enable the vehicle to navigate more safely, covering greater distances at higher speeds. Since the policy training is conducted with MetaDrive in this paper, detailed definitions of state, action and reward related to policy training will be elaborated in following Section 3.2.1.

Autonomous driving requires vehicles to make complex real-time decisions in dynamic environments, presenting two major challenges in policy training: high-dimensional inputs and data efficiency. Autonomous vehicles process rich, multifaceted data including ego state, navigation information, and surrounding information. In scenarios such as lane merging or navigating urban intersections, the system must evaluate critical features like vehicle positions, speeds, and road conditions. However, the complex relationships among these features make feature correlation evaluation difficult, leading to unreliable decision-making. Additionally, in real-world autonomous driving conditions, collecting training data is often time-consuming and expensive, which hinders RL models that typically require large datasets to generalize effectively. Enhancing data utilization is therefore crucial for reducing training times and improving policy robustness.

Therefore, a good policy training method should achieve higher accumulated reward while using fewer training samples because this balance reflects both the efficiency and effectiveness of the learning process. Higher accumulated rewards indicate that the policy is successfully optimizing the desired outcomes, such as safety and speed in the context of autonomous driving. Simultaneously, using fewer training samples demonstrates computational efficiency and practical feasibility, reducing the resources and time required for training. Moreover, a method that requires fewer samples is likely to generalize better, exhibiting robust performance across varied and unforeseen scenarios. Thus, optimizing for both reward and sample efficiency ensures a more scalable, cost-effective, and reliable RL solution.

In this paper, we aim to use Transformer encoder to enhance feature extraction that facilitates RL agent to derive optimal action and adopt REDQ to improve sample efficiency. Detailed information about Transformer encoder and REDQ algorithm will be elaborated in following Sections 3.2.2 and 3.2.3, respectively.

3.2. Proposed methodology

The proposed policy training process is outlined in Fig. 2, encompassing three key components: the MetaDrive environment, the Transformer encoder, and the learning agent. The MetaDrive environment serves as the backdrop for rendering the autonomous driving environment, supplying essential observations such as ego state, navigation details, and surrounding traffic information for informed decision-making by the autonomous policy. The Transformer encoder plays a pivotal role by receiving observations from the MetaDrive

environment, automatically evaluating internal relationships among diverse inputs, and reorganizing the feature vector using its primary self-attention block within the multi-head attention module. This feature vector is then directed to the learning agent module, where it undergoes mapping to actor and critic values within the actor-critic framework. The actor's output guides the generation of actions, specifically throttle/braking and steering angle, through sampling methods. These selected actions are executed within the MetaDrive environment, and the critic network evaluates the actor's performance based on the rewards from the environment. Under the REDQ framework, the actor's performance is refined to maximize cumulative rewards achieved by the learning agent. Further details about each component are elaborated below.

3.2.1. MetaDrive environment

MetaDrive stands out as a comprehensive driving simulation platform designed to construct a broad spectrum of diverse traffic scenarios [11]. Noteworthy is its unique feature of compositionality, enabling the generation of varied driving scenarios through procedural generation and domain randomization. In MetaDrive, every element, ranging from vehicles and obstacles to road structures, is defined as an interactive object with numerous configurable settings constrained within parameter spaces. These settings can be flexibly combined, managed, and actuated, facilitating the seamless composition of new and interactive driving scenarios for research and development purposes.

MetaDrive introduces key features that contribute to its versatility, as illustrated in Fig. 3. This platform possesses the capability to generate an extensive array of road structures by utilizing fundamental roadblocks such as straight sections, ramps, roundabouts, curves, intersections, and more, as depicted in Fig. 3(a). By combining these representative blocks with varying block numbers and random orders, a multitude of diverse road structures can be generated, as shown in Fig. 3(b). This functionality enables the creation of abundant data for training RL agents and facilitates policy evaluation in diverse settings. Additionally, MetaDrive supports widely-used multi-modal observations from RGB camera, depth camera, Lidar, etc, as demonstrated in Fig. 3(c). This provision of multi-modal observations equips the ego agent vehicle with comprehensive information essential for making informed decisions in various autonomous driving scenarios.

In the paradigm of RL, for a specific environment, it needs to define state, action and reward, where state represents the current situation or context, action is a decision or move made by the agent, and reward is numerical feedback indicating the immediate benefit or cost associated with the chosen action in that particular state. For the MetaDrive environment, these crucial elements are defined as follows.

(a) State

MetaDrive furnishes a comprehensive state vector that encapsulates essential information for navigation tasks. This state vector serves as a foundational component in the RL experiments, which comprises three parts, namely ego state, navigation information and surrounding information. The ego state encompasses the current states of the ego vehicle, such as heading, velocity, and relative distance to boundaries. A detailed elucidation of each dimension of the state can be found in MetaDrive documentation [11,55]. The navigation information orchestrates the vehicle's trajectory towards the designated destination. Specifically, MetaDrive initiates this process by computing the route from the spawn point to the ego vehicle's destination. In the original MetaDrive, only checkpoints at lane intersections are placed and the relative distance and direction to subsequent checkpoints are provided. To facilitate training, a series of waypoints are generated along the route at predefined intervals for navigation. The surrounding information is encoded into a vector via Lidar reads. Configurable lasers are used to scan the neighboring area within a radius of 50 m.

(b) Action

MetaDrive receives normalized action $a = [a_1, a_2]^T \in [-1, 1]^2$ as input to control the ego vehicle. At each time step, MetaDrive converts

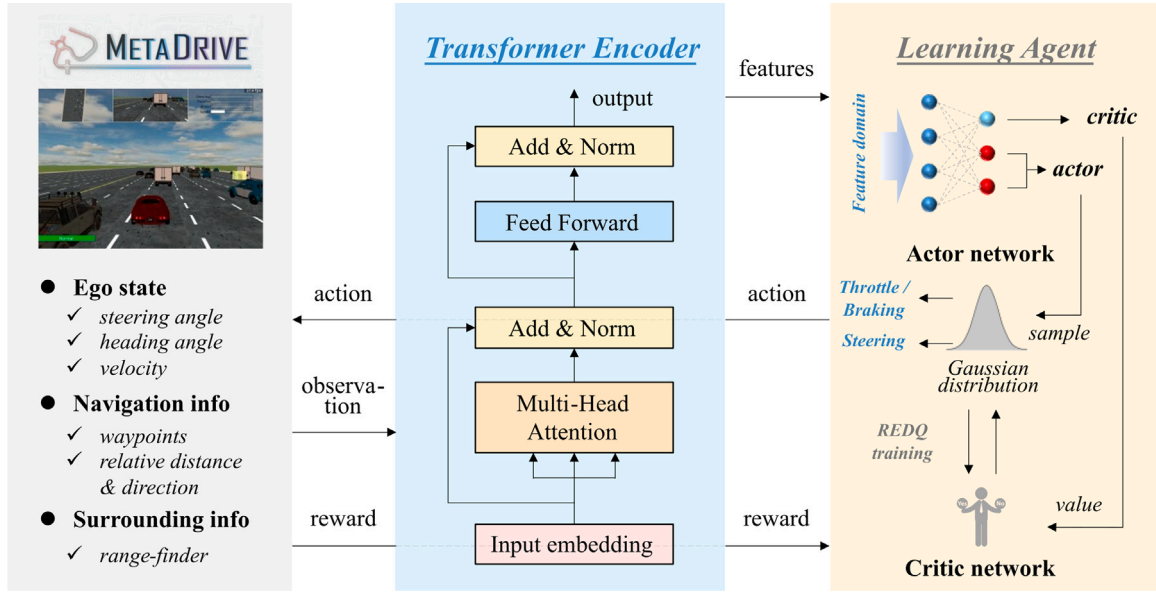


Fig. 2. General framework of the proposed policy training process for the autonomous driving system.

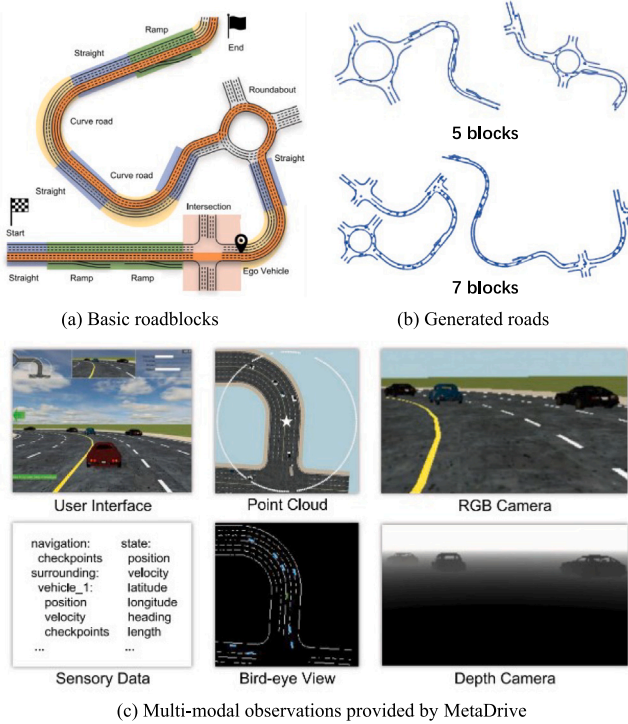


Fig. 3. Key features of MetaDrive simulation environment [11].

the normalized action into the steering u_s , acceleration u_a and brake signal u_b in the following ways:

$$\begin{cases} u_s = S_{\max} a_1 \\ u_a = F_{\max} \max(0, a_2) \\ u_b = -B_{\max} \min(0, a_2) \end{cases} \quad (1)$$

where $S_{\max}$ is the maximal steering angle, $F_{\max}$ is the maximal engine force, and $B_{\max}$ is the maximal brake force.

(c) Reward

The default reward function within MetaDrive is primarily comprised of two components: a dense driving reward and a sparse terminal

reward. The dense reward is determined by the longitudinal movement towards the destination while the sparse reward is given when the episode terminates. In detail, the comprehensive reward function is delineated into three distinct parts, outlined as follows:

$$R = c_1 R_{\text{driving}} + c_2 R_{\text{speed}} + R_{\text{termination}} \quad (2)$$

In the above equation, the driving reward $R_{\text{driving}} = d_t - d_{t-1}$, where d_t and d_{t-1} denote the longitudinal coordinates of the target vehicle in the current lane of two consecutive time steps, providing dense reward to encourage the vehicle to move forward. The speed reward $R_{\text{speed}} = v_t / v_{\max}$ incentives to drive fast, where v_t and $v_{\max}$ denote the current velocity and the maximum velocity, respectively. The termination reward $R_{\text{termination}}$ contains a set of sparse rewards. At the end of the episode, dense rewards will be disabled and only one sparse reward will be given according to the ego vehicle's termination state, which includes three possible situations, namely arriving at the destination, driving out of the road, crashing to other vehicles or obstacles. Three rewards +10, -5, and -5 are set for the above three distinct termination states, respectively. c_1 and c_2 represent two constants that used to scale R_{driving} and R_{speed} . Here, $c_1 = 1.0$, $c_2 = 0.1$.

3.2.2. Feature evaluation using Transformer encoder

The standard Transformer model is primarily composed of an encoder and a decoder [35]. In addressing the autonomous driving control problem, this paper adopts the Transformer encoder as the core of the model. A Transformer encoder block is designed to process and transform input sequences into context-aware representations. It employs a multi-head self-attention (MSA) mechanism to allow each position in the input to consider all other positions, capturing contextual relationships within the sequence. This is followed by a feed-forward neural network (FFN) for further processing. Layer normalization (LN) and residual connections are incorporated to stabilize and enhance the training process by normalizing inputs and maintaining gradient flow. The overall function is to produce a rich, contextual representation of the input data for effective downstream processing. Detailed information of Transformer encoder block can be found in Appendix A.

In the context of policy training for autonomous driving using MetaDrive, the feature extraction process employing a Transformer encoder is illustrated in Fig. 4. Initially, the three components of the state vector – ego state s_A , navigation information s_B , and surrounding information s_C – are each input into three FCNs, namely f_A , f_B and

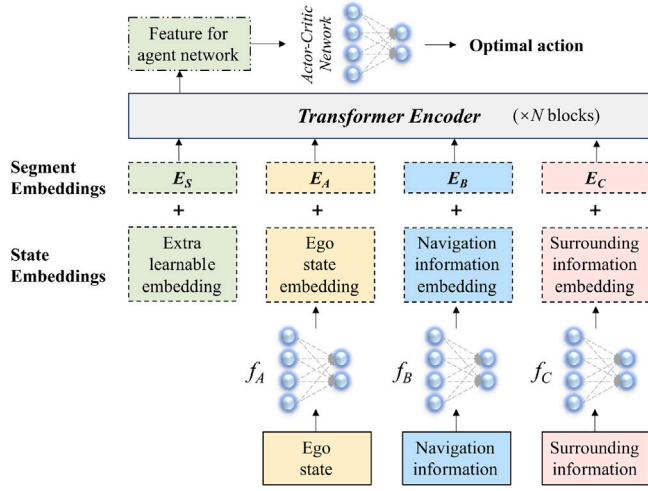


Fig. 4. Process of the feature evaluation using Transformer encoder.

f_C , to standardize their feature vectors to the same length L . This unification is necessary because the Transformer encoder is designed to process inputs of the same attribute with uniform feature lengths. These three networks project the distinct attributes – ego state, navigation information, and surrounding information – into a common feature space, resulting in ego state embedding, navigation information embedding, and surrounding information embedding, respectively, which facilitates subsequent Transformer encoder processing.

An additional learnable embedding x_S is concatenated with the above three state embeddings. This extra embedding interacts with the other embeddings within the Transformer encoder with N blocks, extracting a representative feature of the entire state, akin to the [CLS] token in vision Transformer [56]. Segment embeddings E_S , E_A , E_B and E_C are also added to the original state embeddings to distinguish different state embeddings. Notably, these segment embeddings are learnable, allowing adaptive optimization of the control policy.

The combined state embeddings and segment embeddings z_0 are then input into the Transformer encoder. Only the first output embedding z'_0 , corresponding to the extra learnable embedding, is used as the general feature for the subsequent agent network. This is because, after interacting with all other state embeddings through the Transformer's attention mechanisms, the extra learnable embedding has assimilated the global information from all state embeddings.

The above process can be summarized in mathematical form as follows:

$$z_0 = [x_S; f_A(s_A); f_B(s_B); f_C(s_C)] + [E_S; E_A; E_B; E_C] \quad (3)$$

$$z'_n = \text{MSA}(\text{LN}(z_{n-1})) + z_{n-1}, \quad n = 1, \dots, N \quad (4)$$

$$z_n = \text{FFN}(\text{LN}(z'_n)) + z'_n, \quad n = 1, \dots, N \quad (5)$$

3.2.3. Learning agent

RL can be deemed as adaptive learning by an intelligent agent through a trial-and-error process. The agent and the environment continually interact, with the agent perceiving the environment's state, taking actions to change the environment's state, and receiving corresponding rewards or penalties to guide the update of model parameters. The goal is to achieve the maximum cumulative reward through continuous interaction. Through the cycle of perception-action-evaluation-learning, the intelligent agent continuously acquires knowledge, adjusting and improving its action strategy to adapt to the environment. The interaction between the agent and the environment is commonly

described using the Markov decision process, which consists of a quintuple (S, A, P, r) , representing the state space S , action space A , state transition probability P , and reward function r .

The training objective of an RL model is to find the optimal policy π^* that maximizes the expected cumulative reward, namely:

$$\pi^* = \arg \max_{\pi} E_{(s_t, a_t) \sim \rho_{\pi}} \left[\sum_t r(s_t, a_t) \right] \quad (6)$$

where ρ_{π} represents the distribution of state-action trajectories formed by the policy π . However, for the high-dimensional non-convex and non-linear optimization problem formed by the autonomous driving system, the model is prone to getting stuck in local optima. Additionally, the model's performance is highly sensitive to hyperparameters and neural network structures. Therefore, to enhance the exploration capability of the RL model, this paper adopts SAC as the RL basement. Detailed information about SAC algorithm can be found in Appendix B.

In the conventional SAC algorithm, the UTD is equal to 1, meaning that for each interaction with the environment, only a single update is applied to the policy parameter. This results in a significant reduction in data utilization rate. To achieve a larger UTD and address this limitation, the REDQ algorithm is introduced. The pseudocode for REDQ is outlined in Table 1, and it is compatible with various standard off-policy model-free RL algorithms. In this paper, SAC is employed. REDQ comprises the following key components. To enhance sample efficiency, the UTD ratio G is set to a value much greater than one. To mitigate variance in the Q-function estimate, REDQ utilizes an ensemble of N Q-functions, each randomly and independently initialized but updated with the same target. To alleviate over-estimation bias, the Q-function's target involves minimization over a random subset M of the N Q-functions. The subset's size, denoted as M , remains fixed and is referred to as the in-target minimization parameter. Since the default choice for M is $M = 2$, the algorithm is referred as double Q-learning.

4. Validation

In this section, the effectiveness of the proposed method is validated. The section begins with a comprehensive description of the general performance of the method. Following this, the feature evaluation performance of the Transformer encoder block is analyzed across five representative driving scenarios and compared with other commonly used feature extraction networks. Finally, a comparison between the REDQ method and traditional RL algorithms is conducted, highlighting how the introduced high UTD ratio can enhance data sample efficiency and how the ensemble strategy and in-target minimization in REDQ can improve training stability.

4.1. General performance

The training process is conducted on a computer with a CPU of AMD Ryzen 9 5950X and a graphic card of Nvidia RTX 3080. All associated learning algorithms and scripts are programmed in Python. For the Transformer encoder, four identical layers are concatenated and the head number in multi-head attention is set as 4. The feature dimension for the Transformer encoder is set as 256. The dimension of FFN in the Transformer encoder is set as 1024, which is four times of feature length as recommended in original Transformer architecture.

The hyperparameters used for the REDQ algorithm are listed in Table 2. The fine-tuning process for optimizing the hyperparameters is realized by leveraging the Optuna library [57]. It involves defining a hyperparameter search space that includes key parameters such as discounting factor, learning rate, batch size, etc. The objective function of reward is used to train the model with these hyperparameters and evaluate its performance across several episodes. By initializing an Optuna study aimed at maximizing the reward metric and running multiple optimization trials, the optimal set of hyperparameters that enhance the RL agent's performance can be identified.

Table 1

Detailed implementation of REDQ algorithm [52].

Algorithm: Randomized Ensembled Double Q-learning (REQD)	
1:	Initialize policy parameters θ , N Q-function parameters $\phi_i, i=1, \dots, N$, empty replay buffer D . Set target parameters $\phi_{\text{targ}, i} \leftarrow \phi_i$
2:	repeat
3:	Take one action $a_i \sim \pi_\theta(\cdot s_i)$. Observe reward r_i , new state s_{i+1} .
4:	Add data to buffer: $D \leftarrow D \cup \{(s_i, a_i, r_i, s_{i+1})\}$
5:	for G updates do
6:	Sample a mini-batch $B = \{(s, a, r, s')\}$ from D
7:	Sample a set $\mathcal{M}$ of M distinct indices from $\{1, 2, \dots, N\}$
8:	Compute the Q target y (same for all of the N Q-functions): $y = r + \gamma (\min_{i \in \mathcal{M}} Q_{\phi_{\text{targ}, i}}(s', \tilde{a}) - \alpha \log \pi_\theta(\tilde{a}' s'))$, $\tilde{a} \sim \pi_\theta(\cdot s')$
9:	for $i = 1, \dots, N$ do
10:	Update ϕ_i with gradient descent using $\nabla_{\phi_i} \frac{1}{ B } \sum_{(s, a, r, s') \in B} (Q_{\phi_i}(s, a) - y)^2$
11:	Update target networks with $\phi_{\text{targ}, i} \leftarrow \rho \phi_{\text{targ}, i} + (1 - \rho) \phi_i$
12:	Update policy parameters θ with gradient ascent using $\nabla_{\theta} \frac{1}{ B } \sum_{s \in B} \left(\frac{1}{N} \sum_{i=1}^N Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s) s) \right), \tilde{a}_\theta(s) \sim \pi_\theta(\cdot s)$

Table 2

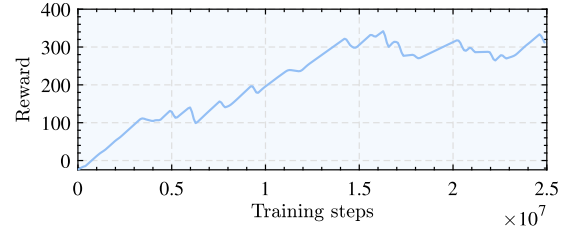
Hyperparameters for SAC-based REDQ algorithm.

Hyperparameters	Value
Number of updates G in REDQ	20
Ensemble number N in REQD	10
Steps before learning start	10 000
Discounting factor γ	0.99
Parameter for soft update of the target network τ	0.005
Entropy regularization coefficient α	0.2
Number of critic updates before an actor update	20
Train batch size	256
Penalty learning rate	0.01

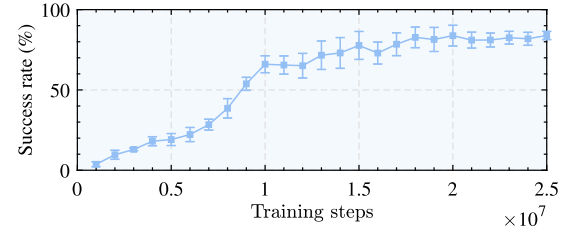
MetaDrive has the capability to generate diverse traffic scenes by adjusting the random seed during the instantiation of the MetaDriveEnv class. In this context, we specify the *start_seed* as 1000 and *num_scenarios* as 1000 to create a pool of 1000 distinct scenes for training the RL agent. During the evaluation, we set the *start_seed* to 0 and *num_scenarios* to 100, producing 100 testing traffic scenes that differ from the training set for assessing the agent's generation capabilities. The evaluation is conducted on the 100 testing traffic scenes for 10 rounds.

The reward with respect to the training steps is shown in Fig. 5(a), where the REDQ algorithm's performance displays a positive trend in rewards over training steps. With rewards scaling up to over 300 and the graph showing a general upward trajectory, the algorithm demonstrates an improvement in policy. Despite the fluctuations and subsequent dip around the mid-point of training, the algorithm recovers and stabilizes towards the end. The total training steps are 2.5×10^7 for the algorithm to achieve successful learning and adaptation.

To assess the performance of the trained agent, it is essential to establish evaluation metrics. In this case, alongside reward, the success rate is also employed, which represents the proportion of episodes where the agent successfully reaches its destination. Unlike episodic reward, the success rate proves to be a more suitable measure for assessing generalization, particularly given the multitude of scenes with varying properties such as road length and traffic density. This variation across scenes can lead to fluctuations in reward, making the success rate a more robust evaluation criterion. During the training process, we saved the trained model every 1×10^6 steps, therefore total of 25 models are obtained ultimately when the 2.5×10^7 training steps terminate. The success rate of the 25 saved models on the testing dataset are shown in Fig. 5(b), where the error bar extending from each average success rate data point indicates variability or uncertainty as each model is evaluated 10 times. It can be seen that the graph shows an overall increasing trend, meaning that the success rate improves as the number of training steps increases. Towards the end, the success rate comes to plateau, suggesting the model has converged. It is also interesting to note that the variance of the model shrinks as the training step increases from 1.5×10^7 to 2.5×10^7 . Referring to Fig. 5(a), the



(a) Reward trend during training



(b) Success rate on testing dataset with different training steps

Fig. 5. Key metrics for the agent training.

reward during this stage has no significant increase. It can be concluded that the training close to the end makes the model more robust and has better generalization ability despite little reward increase.

4.2. Analysis of Transformer encoder function

To elucidate the Transformer encoder's proficiency in feature evaluation, Fig. 6 delineates the performance of the proposed method across five distinct driving scenarios: straight lane, approaching roundabout, in roundabout, collision avoidance, and lane change. This illustration includes both the resultant actions and the weight distribution assigned by the Transformer encoder to various observation inputs. For instance, in a straight lane scenario, the resultant action parameters $a_1 = 0.66$, $a_2 = 0.01$ indicate a tendency for the vehicle to accelerate and maintain a straight trajectory, according to the action definitions outlined in Section 3.2.1(b).

In the straight lane scenario, the contribution percentages of the ego state, Lidar information, and navigation information to the acceleration action are 41%, 50%, and 9% respectively, while for the steering action, they are 16%, 32%, and 52% respectively. These weights are computed through a process that begins with calculating the similarity between the query and keys across all feature inputs in the Transformer encoder layer. The weights from all layers are then integrated with those from the fully connected network, which outputs the mean action.

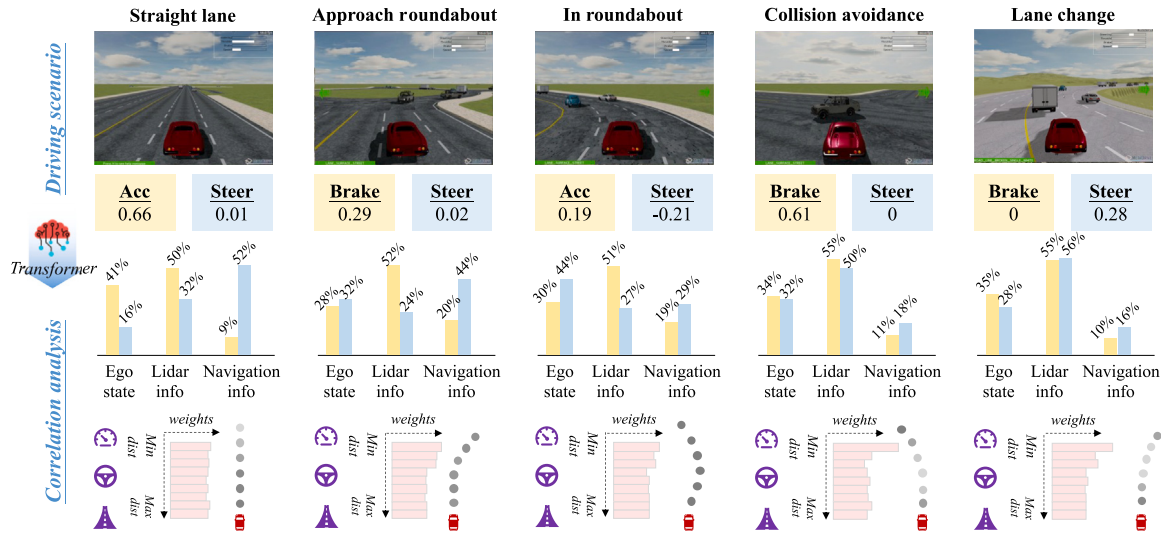


Fig. 6. Visualization of Transformer encoder weights for different feature inputs in representative driving scenarios.

Subsequently, these averaged weights from different input segments are normalized to sum to one. The detailed weights for each feature within a group are derived similarly, and the weights for the two actions (acceleration and steering) are combined with equal consideration.

The results indicate that acceleration is primarily influenced by the ego state and Lidar information, whereas steering decisions are more heavily based on Lidar and navigation information. This aligns with the expectation that longitudinal control (acceleration/braking) should consider the vehicle's current state to avoid abrupt changes. Lateral control (steering), on the other hand, involves both obstacle avoidance based on Lidar reads and adherence to navigational waypoints.

Regarding the specific contributions of individual features to the decision-making process in autonomous driving, an analysis of the straight lane scenario reveals a relatively equal influence of speed, steering angle, and relative distance to boundaries on the final action. This is evidenced by the uniformity in color intensity within the visual representation in the figure. The weight distribution remains consistent across the range of distances detected by the Lidar sensor. In the context of a straight lane, where the presence of proximate obstacles is minimal, the minimum and maximum distances reported by the Lidar tend to be similar, leading the Transformer encoder to assign comparable weights to this data. In terms of navigation information, specifically the tracking of waypoints, the Transformer encoder exhibits a gradient of attention, focusing more on waypoints in closer proximity to the vehicle's current position and progressively less on those further away. This pattern is visually indicated by a gradation in color, with darker shades near the vehicle. This allocation of attention aligns with intuitive navigation strategies, where waypoints immediately ahead of the vehicle are prioritized for immediate route planning and decision-making.

In the second scenario where the vehicle approaches a roundabout, it initiates braking to decelerate before entering the curved roadway. In this context, navigation information assumes a heightened role in the vehicle's longitudinal control compared with the straight lane scenario. The necessity for braking is primarily attributed to the transition from a straight lane to a curved path. Regarding lateral control, the influence of the ego state becomes more pronounced compared to the straight lane scenario, likely due to the imminent turning maneuver. In this phase, the weights attributed to various features within the ego state remain relatively balanced. However, Lidar data indicating shorter obstacle distances receives increased focus, reflecting the need for heightened awareness of nearby vehicles when navigating into the roundabout. Unlike in straight lane driving, where distant waypoints receive less emphasis, the approach to a roundabout necessitates a more uniform consideration of all waypoints. This equal weighting stems

from the changing trajectory, as the vehicle transitions from a straight to a curved path. The properties of the upcoming curved road exert a significant influence on the current braking decisions, necessitating a comprehensive evaluation of the entire trajectory, including more distant elements of the route.

In the third scenario where the vehicle navigates a turn within a roundabout, it exhibits a slight acceleration and initiates a leftward steering maneuver. The observed control behavior in this context shows notable parallels to the approaching roundabout scenario, with some distinct variations. A key difference is observed in the relative contributions of navigation information and ego state to the vehicle's control decisions. Specifically, the prominence of navigation information diminishes, whereas the influence of the ego state is augmented in this scenario. This shift can be attributed to the dynamics of navigating a roundabout, characterized by a consistent road curvature. In such a setting, the control policy appears to prioritize maintaining the vehicle's state stability. This entails a greater reliance on the vehicle's current dynamics, as represented by the ego state, to ensure a smooth and stable trajectory through the roundabout. The reduced emphasis on navigation information suggests a lesser need for external navigational cues once the vehicle is effectively within the roundabout's constant curvature environment, focusing instead on sustaining a steady driving state.

In the fourth scenario of collision avoidance, the ego vehicle executes a sharp braking maneuver in response to a jeep intersecting its path. Within this high-stakes situation, both the ego state and Lidar information significantly influence the vehicle's longitudinal and lateral controls. The urgency of the scenario results in a diminished role for navigation information, as adhering strictly to predefined waypoints becomes secondary to immediate safety concerns. A notable aspect of this scenario is the disproportionately high weight assigned to the minimum distance data from Lidar sensor. This prioritization reflects the critical importance of the nearest obstacle – in this case, the crossing jeep – in determining the urgency and extent of the braking action. The immediacy of the threat overrides other considerations, making the closest obstacle distance the primary determinant of the vehicle's responsive action. Furthermore, an analysis of waypoint contributions in this scenario reveals a strategic deprioritization of waypoints proximal to the jeep, acknowledging the vehicle's potential deviation from the planned route for safety. Conversely, waypoints located further from the immediate danger area receive greater attention. This distribution of weights ensures that, while immediate safety is prioritized, the vehicle remains oriented towards its longer-term navigational goals,



Fig. 7. Visualization of attention maps in different layers of Transformer encoder.

preparing to resume its intended trajectory once the immediate hazard is navigated.

In the last lane-change scenario, the ego vehicle turns right to avoid a collision with the van ahead. The feature analysis result in this scenario is similar to that of collision avoidance, where Lidar information predominantly influences the vehicle's maneuvering decisions, with the Lidar sensor detecting the minimum distance playing a crucial role. The vehicle's significant steering angle facilitates its lane change from the left lane to the center lane, effectively avoiding the collision of the front van. In this situation, braking is not applied because the vehicle is navigating a roundabout, where its low speed ensures a safe lane change. This also explains why the ego state contributes moderately to the final driving decision. Regarding the navigation information, the original waypoints were generated based solely on the road structure without accounting for dynamic obstacles. Following these waypoints would have led to a collision with the van. Thanks to the strong feature evaluation capability of Transformer encoder, the vehicle can deprioritize navigation information and disregard it when executing a lane change, prioritizing safety instead.

To further explore the working mechanisms of the Transformer encoder, Fig. 7 visualizes the attention maps of different heads in the self-attention module across various Transformer encoder layers for a specific scene. Each row in the figure represents a layer, while each column represents a head. In each attention map, both the rows and columns represent different feature domains: learnable embedding, ego state embedding, navigation information embedding, and Lidar information embedding. Each row represents a query and each column represents a key, with the sum of each row in the attention map being 1, indicating that the total attention weight for all keys to a given query is normalized to 1.

First, comparing different columns reveals that the attention weights for different heads are significantly distinct. This distinction arises because different heads in the self-attention module are designed to extract diverse features from various subspaces, thus enabling the RL agent to comprehensively observe the state from multiple perspectives.

From layer 1 to layer 4, the cross-head similarity, calculated as the averaged dot product, decreases from 0.2622 to 0.2398. This reduction in similarity indicates that the model maintains diverse representations

in deeper layers, which is advantageous for capturing different feature aspects of the data.

Moreover, the average variance of attention weights for the Transformer encoder layers also decreases, with values of 0.0519, 0.0217, 0.0205, and 0.0142 from layer 1 to layer 4, respectively. As the network delves deeper, the model progressively refines its understanding of the input, focusing on more specific and consistent patterns, leading to a reduction in the variance of attention weights. This trend suggests that the model is converging on certain key pieces of information crucial for the final autonomous driving task.

The concurrent decrease in both cross-head similarity and variance suggests that the model's attention mechanisms are becoming more consistent, with the heads converging towards focusing on different feature aspects. This observation is particularly reasonable in well-trained models, where deeper layers are expected to integrate and refine information from earlier layers effectively.

It can be seen from the above analysis that thanks to the attention mechanism, the trained Transformer encoder network could give desirable weights to different input features in different driving scenarios so that the vehicle could be highly adaptive to cope with diverse and complex autonomous driving tasks.

To underscore the superior feature evaluation capabilities of the Transformer encoder, we train the RL agents with other neural networks, namely FCN, RNN and CNN-LSTM and test these agents over 100 testing scenarios for 10 rounds. The comparative results, detailed in Table 3 and demonstrated in Fig. 8, demonstrate that the Transformer encoder achieves the highest reward and success rate. Specifically, the Transformer encoder shows improvements in absolute reward by 131.2, 107, and 61.6 over FCN, RNN, and CNN-LSTM, respectively. Similarly, in terms of success rate, the Transformer encoder exhibits enhancements of 19.6%, 21.4% and 13.6% compared to FCN, RNN, and CNN-LSTM, respectively. Furthermore, as illustrated in Fig. 8, the Transformer encoder demonstrates a more rapid increase in both reward and success rate during the initial stages of training compared to the other three networks. This can be explained by the fact that unlike FCN, RNN and CNN-LSTM, which rely heavily on convolutions and sequential processing, respectively, the Transformer encoder employs self-attention mechanisms that enable it to capture dependencies

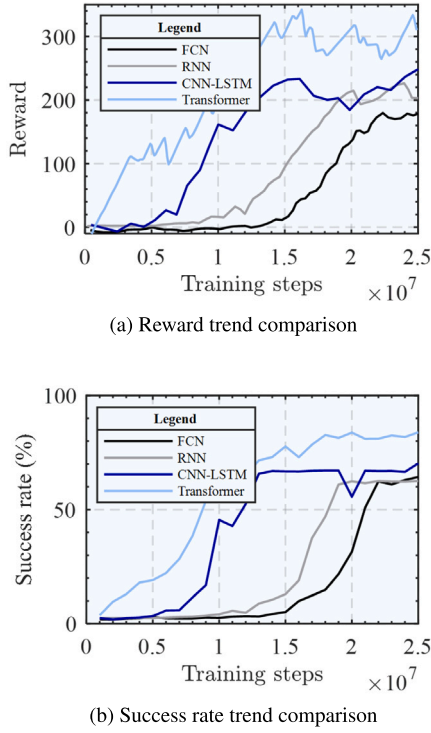


Fig. 8. Comparison of different neural networks in the training process.

Table 3

Comparison of performance using different neural networks.

	FCN [32]	RNN [34]	CNN-LSTM [33]	Transformer encoder
Reward	178.6	202.8	248.2	309.8
Successrate	64.3%	62.5%	70.3%	83.9%

and contextual information more effectively. This self-attention allows the Transformer to weigh the relevance of different input features dynamically, providing a more nuanced understanding of complex data patterns, as revealed in Figs. 6 and 7. By leveraging these advantages, the Transformer encoder achieves superior performance in tasks requiring detailed feature extraction and understanding like autonomous driving, setting a new benchmark for model efficiency and accuracy in the field of deep learning.

4.3. Analysis of REDQ algorithm

The hyperparameter N , representing the number of Q-functions, is critical for the REDQ algorithm because it balances the bias-variance trade-off in value estimation, thereby enhancing the overall stability and performance of the learning process. To compare the performance

Table 4

Statistics of REDQ with different numbers of Q-functions at the end.

	$N = 2$	$N = 5$	$N = 10$	$N = 15$
Reward	123.9	203.4	309.8	297.0
Average normalized bias	-0.3327	-0.2365	-0.1848	-0.2220
Standard deviation of normalized bias	0.2302	0.1132	0.0850	0.0821

of REDQ under different settings of N , the bias in Q-function estimation can be analyzed. Consider $Q^\pi(s, a)$ as the action-value function for a policy π using the standard infinite-horizon discounted return definition and let $Q_\phi(s, a)$ be an estimate of $Q^\pi(s, a)$, defined as the average of $Q_{\phi_i}(s, a)$, $i = 1$ to N , when using an ensemble. The bias of an estimate at a state-action pair (s, a) is defined as $Q_\phi(s, a) - Q^\pi(s, a)$. To assess this bias, episodes from various random initial states under the current policy π can be conducted and then the average and standard deviation of the Q-value bias distribution can be calculated. The average provides insight into whether Q_ϕ is generally overestimating or underestimating, while the standard deviation (std) measures the uniformity of bias across different state-action pairs. Given the significant variability in Monte Carlo return values throughout training, the normalized bias of the estimate $Q_\phi(s, a)$ is introduced, which is defined as $(Q_\phi(s, a) - Q^\pi(s, a)) / |E_{\tilde{s}, \tilde{a} \sim \pi} [Q^\pi(\tilde{s}, \tilde{a})]|$. This normalization involves dividing the bias by the absolute value of the expected discounted Monte Carlo return for state-action pairs sampled from the current policy. The focus on normalized bias helps illustrate the magnitude of the bias relative to the scale of the current Monte Carlo return.

Fig. 9 and Table 4 illustrate the reward, average normalized bias, and std of normalized bias under various N settings. The data indicate that settings of $N = 10$ and $N = 15$ achieve higher rewards within the limited 12.5×10^5 interactions compared to settings of $N = 2$ and $N = 5$. This is attributed to the fact that using more Q-functions facilitates greater exploration when the agent searches for the optimal action, thus improving performance with fewer interactions. However, a larger N results in a significantly higher computational burden, with performance gains saturating beyond a certain point. Given that both $N = 10$ and $N = 15$ yield similar performance and that the average and standard deviation of the normalized bias for both settings approach zero towards the end, $N = 10$ is chosen as the default setting for the analysis.

Moreover, Fig. 9(a) reveals an interesting phenomenon: when $N = 2$, the reward remains at a low level for a prolonged period. This can be attributed to primacy bias, as described in Ref. [58,59]. Although a high UTD ratio enhances the utilization of interaction samples, it can also lead to overfitting to limited experiences. To address this issue, we follow the approach in Ref. [60], which involves sampling a number of random datapoints at the beginning of training to mitigate primacy bias. Fig. 9(a) shows that when N exceeds 5, primacy bias is significantly reduced, as evidenced by the absence of a prolonged plateau at the start of training. A larger N setting also helps to alleviate primacy bias by introducing more randomness into the Q-target, functioning

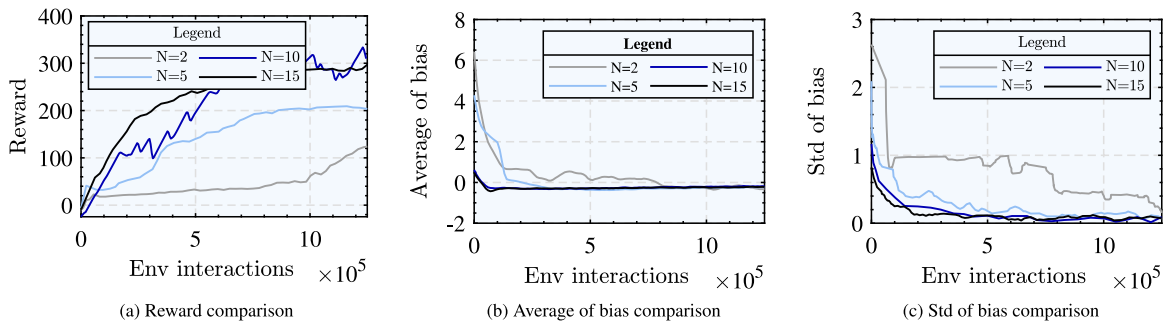
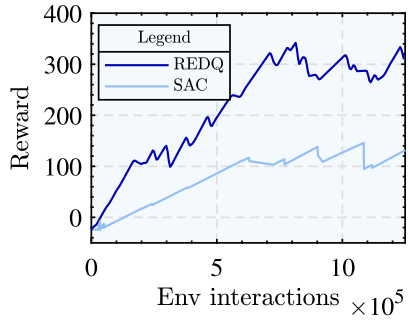
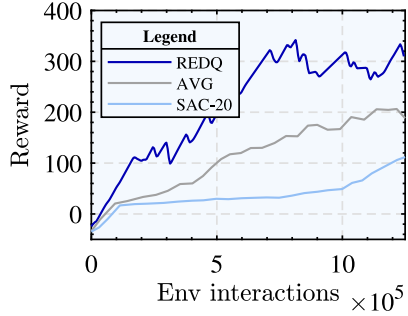


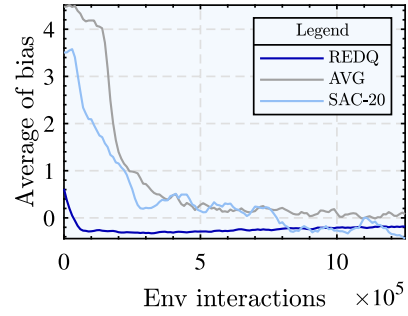
Fig. 9. Comparison of REDQ with different numbers of Q-functions.



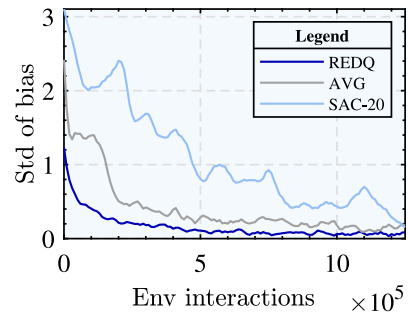
(a) UTD influence



(b) Reward comparison



(c) Average of bias comparison



(d) Std of bias comparison

Fig. 10. Comparison between REDQ and other algorithms.

similarly to random sampling. Therefore, both random sampling and larger N settings effectively address the primacy bias problem.

In order to demonstrate the sample efficiency of the REDQ algorithm, it is first compared with the vanilla SAC algorithm and the result is shown in Fig. 10(a). It needs to be highlighted here that the x -axis represents number of environment interactions rather than number of

Table 5

Performance of REDQ, AVG and SAC-20.

	REDQ	AVG	SAC-20
Reward	309.8	188.2	111.9
Average normalized bias	-0.1848	0.0796	-0.4025
Standard deviation of normalized bias	0.0850	0.1769	0.1504

training steps as in Fig. 5. Because the total number of training steps is 2.5×10^7 and $UTD = 20$ here, the number of environment interactions equals $2.5 \times 10^7 / 20 = 12.5 \times 10^5$. For comparison, vanilla SAC with $UTD = 1$ is run for the same environment interactions.

Fig. 10(a) demonstrates the superior effectiveness of the REDQ algorithm over SAC in terms of achieving higher rewards at a faster rate through learning from environmental interactions. To achieve a reward of 100, SAC requires approximately 5.55×10^5 interactions, whereas REDQ requires only 1.57×10^5 interactions, indicating that REDQ achieves a training acceleration approximately 3.5 times faster than SAC. After 12.5×10^5 interactions, REDQ attains an episode reward of 309.8, which is more than double the 126.9 achieved by vanilla SAC. This suggests that REDQ's approach to recycling environment interaction data allows for more comprehensive exploitation of data value, thereby enhancing sample efficiency. However, it is important to note that the increase in rewards is not directly proportional to the UTD ratio. This is because the additional value derived from recycling data is limited. Doubling the data recycling does not equate to a similar magnitude of improvement as would be achieved with a dataset twice the size. This principle of sample efficiency is particularly vital in the context of autonomous driving. If RL is to be applied in real-world scenarios, autonomous vehicles must demonstrate the ability to learn continuously and adapt to new environments or behaviors. Sample-efficient learning enables quicker adaptation to changes with less data, ensuring that autonomous driving systems can stay current with minimal data requirements.

In order to highlight the importance of ensemble and in-target minimization in REDQ, it is compared with SAC-20 and AVG, where SAC-20 is SAC but with UTD increases from 1 to 20 but no ensemble and in-target minimization is involved. AVG has an ensemble to estimate the Q-function but the estimation is conducted over the whole ensemble rather than the subset as in REDQ.

Fig. 10(b)–(d) and Table 5 compare the performance of the above three algorithms on the generalization task of MetaDrive. It can be seen that REDQ demonstrates a notably accelerated learning pace compared to both SAC-20 and AVG as shown in Fig. 10(b). Particularly noteworthy is REDQ's remarkably low normalized average of bias throughout the majority of the training process in Fig. 10(c). This implies that REDQ could realize accurate Q-function estimation in complex MetaDrive environment. However, in the case of SAC-20 and AVG, the elevated UTD ratio results in an average bias that undergoes fluctuations throughout the training process. Additionally, for the majority of the training duration, REDQ exhibits a consistently small and stable bias as shown in Fig. 10(d), while a high normalized std of bias exists for SAC-20, indicating that the bias is highly non-uniform. Compared with SAC-20, REDQ achieves a 43% reduction in the std of bias at the end of training. For AVG, thanks to its introduction of ensemble, its std of bias is greatly reduced and is only a little higher than that of REDQ. Combining the observation from Fig. 10(c) and (d), it can be concluded that the ensemble is greatly helpful in lowering the std of bias while the in-target minimization is beneficial for depressing the fluctuation and magnitude of average bias. By introducing both ensemble and in-target minimization, REDQ achieves faster and more stable convergence performance in MetaDrive training.

To underscore the superiority of REDQ, comparative analyses were conducted with two widely used RL algorithms, specifically TD3 and TRPO. Initially, TD3 and TRPO were evaluated with a UTD ratio of 1. To explore enhanced performance, the UTD ratios for these algorithms

Table 6
Performance comparison among different reinforcement learning methods.

Algorithms	Reward	Success rate
TRPO	100.8	14.9%
TRPO-20	101.3	15.6%
TD3	106.2	16.3%
TD3-20	108.7	16.4%
SAC	126.9	16.4%
SAC-20	111.9	16.2%
REDQ	309.8	83.9%

were increased to 20, resulting in TD3-20 and TRPO-20, respectively. SAC and SAC-20 were also included in the comparisons. Table 6 presents the final reward and success rate metrics for these methods.

It can be seen that REDQ consistently outperforms these algorithms in both reward attainment and success rates. Specifically, REDQ exhibits an absolute improvement of 182.9–209 in reward and 67.5%–69% in success rate over the other methods. Notably, our findings reveal that simply increasing the UTD ratio does not lead to beneficial improvements in the MetaDriving autonomous driving task. This ineffectiveness stems from the amplified mean and standard deviation of bias, as depicted in Fig. 10(c) and (d). Particularly, SAC-20 shows significantly higher standard deviations in bias during early stages compared to REDQ, adversely affecting training performance. The above observation underscores the critical role of incorporating multiple Q-functions and in-target minimization mechanisms within REDQ, emphasizing their importance in achieving superior RL outcomes.

5. Conclusion

This paper introduces a novel approach where a Transformer encoder is applied to assess the significance of features within high-dimensional inputs specific to autonomous driving scenarios while REDQ, characterized by a high UTD ratio, is adopted to maximize data utilization and enhance sample efficiency. The core findings of this study are summarized as follows.

- Leveraging the self-attention mechanism inherent in the Transformer encoder module, the proposed method adeptly assigns suitable weights to various observational inputs across different autonomous driving scenarios. In these scenarios, the longitudinal control and lateral control vary in response to different factors such as lane keeping, vehicle stability, or the need for obstacle avoidance. Compared with commonly used feature extraction networks, the Transformer encoder shows superior performance thanks to its self-attention mechanism.
- REDQ enhances data utilization efficiency via iterative training, and it attains a reduced average bias and stabilizes the standard deviation of bias through the implementation of ensemble strategies and in-target minimization techniques. This approach results in almost three times the rewards of the SAC, TRPO and TD3 algorithms for the same level of environment interactions, and it demonstrates significantly lower bias variability when compared to SAC-20 and AVG.

In future work, we aim to expand our research in two key directions. Firstly, we plan to incorporate multi-modality data, including RGB and semantic images from MetaDrive, as input for training the RL agent. This approach is intended to provide the RL agent with more comprehensive information about the surrounding environment, facilitating smarter decision-making. Existing research indicates that the Transformer architecture can handle multi-modality data without significant modifications, suggesting that the control architecture presented in this paper can be effectively adapted for such data. Secondly, we intend to implement the proposed method in an actual vehicle to validate its effectiveness. By utilizing a diverse array of driving scenarios collected from real-world conditions to train the RL agent, we expect to significantly enhance the model's generalization capability, thereby increasing its robustness and adaptability to various driving environments.

CRediT authorship contribution statement

Jie Fan: Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization. **Xudong Zhang:** Project administration, Funding acquisition, Conceptualization. **Yuan Zou:** Validation, Supervision. **Yuanyuan Li:** Visualization, Validation, Conceptualization. **Yingqun Liu:** Visualization, Validation. **Wenjing Sun:** Visualization.

Declaration of competing interest

We declare that we have no financial and personal relationships with other people or organizations that can inappropriately influence our work, there is no professional or other personal interest of any nature or kind in any product, service and/or company that could be construed as influencing the position presented in, or the review of, the manuscript.

Acknowledgments

This work is supported by National Natural Science Foundation of China (No. 52472393), High-end Foreign Experts Recruitment Program, China (No. G2023178009L), Beijing JinQiao Project, China and Graduate Program for Basic Research Skills and Innovation Capability Enhancement from Beijing Institute of Technology, China. Authors would also like to thank anonymous reviewers for their constructive suggestions.

Appendix A. Basis of transformer encoder

The standard Transformer encoder includes multi-head attention layers, feedforward networks, residual connections, and layer normalization. The feature matrix undergoes residual connections and layer normalization after passing through the multi-head attention layers and feedforward network layers. Residual connections effectively address the issues of gradient vanishing and network degradation [61]. The role of layer normalization is to accelerate training speed and model convergence [62].

The multi-head attention mechanism is the core of the Transformer encoder, as shown in Fig. 11. It divides the model into multiple heads, forming different subspaces that allow the model to focus on various aspects of information. In the end, information from different aspects is integrated, aiding the network in capturing richer feature information.

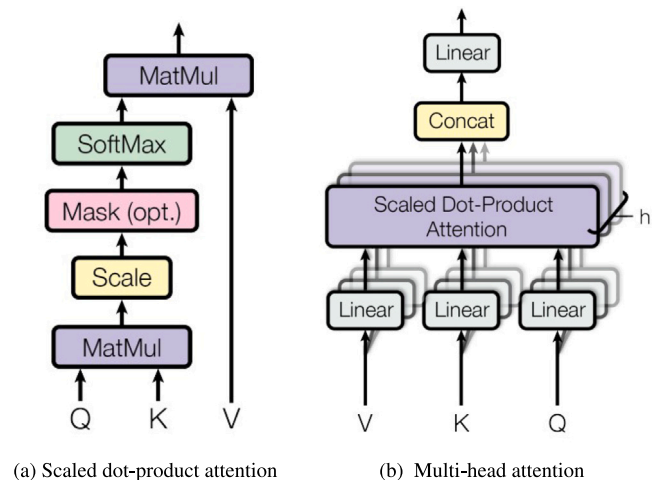


Fig. 11. Attention mechanism in Transformer encoder layer [35].

Here the scaled dot-product attention is adopted and the computational formulas are shown in Eqs. (7) to (10).

$$\begin{cases} Q = W^Q X \\ K = W^K X \\ V = W^V X \end{cases} \quad (7)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (8)$$

$$H_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right) \quad (9)$$

$$\text{MultiHead}(Q, K, V) = [H_1, \dots, H_h] W_o \quad (10)$$

where the query matrix Q , key matrix K , and value matrix V are generated by transforming the feature vector matrix X . W^Q , W^K , and W^V are all linear transformation matrices. d is the scaling factor. W_i^Q , W_i^K , and W_i^V are the transformation matrices projecting Q , K , V into the i th subspace, where i ranges from 1 to h , and h is the total number of subspaces. H_i represents the single-head attention values in the i th subspace. W_o is the transformation matrix used to concatenate the attention values from all subspaces.

Appendix B. Soft actor–critic algorithm

The objective function of SAC algorithm is expressed as [63]:

$$\pi^* = \arg \max_{\pi} E_{(s_t, a_t) \sim \rho_{\pi}} \left\{ \sum_t r(s_t, a_t) + \alpha H[\pi(\cdot | s_t)] \right\} \quad (11)$$

where H is the entropy term for taking actions under state s_t , and α is the temperature coefficient representing the proportion of exploration capability. It can be observed that the objective function of this method consists of two components: the sum of cumulative rewards (the first term) and the entropy regularization term (the second term). The entropy regularization term aims to maximize the entropy of the action distribution chosen at a particular state s_t . Therefore, while the agent pursues the maximization of cumulative rewards, it is also encouraged to explore more in the action space, avoiding premature convergence to local optima. The modified Bellman equation is then expressed as:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma E_{s_{t+1}, a_{t+1}} \{Q(s_{t+1}, a_{t+1}) - \alpha \ln[\pi(a_{t+1} | s_{t+1})]\} \quad (12)$$

$$V(s_t) = E_{a_t \sim \pi} [Q(s_t, a_t) - \alpha \ln \pi(a_t | s_t)] \quad (13)$$

where γ is the reward discount factor, and $V(s_t)$ represents the state-value function.

It can be proven that the improved policy distribution is directly proportional to the exponential form of the Q-function. The resulting action-value function in this case is referred to as the soft Q-value function, expressed as:

$$\pi(a_t | s_t) \propto \exp[Q(s_t, a_t)] \quad (14)$$

During the process of policy improvement for the intelligent agent, the objective is to make the policy tend towards the exponential form of the Q-value function. In this context, the policy update is performed by minimizing the Kullback–Leibler (KL) divergence, expressed as:

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left\{ \pi'(\cdot | s_t) \left| \frac{\exp\left[\frac{1}{\alpha} Q^{\pi'}(s_t, \cdot)\right]}{Z^{\pi'}(s_t)} \right. \right\} \quad (15)$$

where Π represents the set of policy distributions (here Gaussian distribution is adopted), and $Z^{\pi'}(s_t)$ denotes the normalization constant.

To enhance the adaptive learning and generalization capabilities of the model, deep neural networks are employed to represent the Q-value function and policy distribution. The parameter learning of the Q-value network is achieved by minimizing the residual $J_Q(\theta)$, expressed as:

$$J_Q(\theta) = E_{(s_t, a_t, s_{t+1}) \sim D, a_{t+1} \sim \pi_{\phi}} \left\{ \frac{1}{2} [Q_{\theta}(s_t, a_t) - r(s_t, a_t) - \gamma Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) + \alpha \gamma \log(\pi_{\theta}(a_{t+1} | s_{t+1}))]^2 \right\} \quad (16)$$

where θ represents the parameters of the Q-value network, ϕ represents the parameters of the policy network, and $\bar{\theta}$ denotes the parameters of the target Q-value network. The update is performed during training using the following equation:

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta} \quad (17)$$

where τ represents the updating parameter. In practice, to enhance the stability of training, two Q-value networks are introduced, and in each iteration, the one with the smaller value is selected.

For the update of the policy network, namely minimizing the KL divergence in Eq. (15), the constant term $Z^{\pi}(s_t)$ is removed and coefficient α is multiplied, resulting in the transformed expression:

$$J_{\pi}(\phi) = E_{s_t \sim D, a_t \sim \pi_{\phi}} [\alpha \ln(\pi_{\phi}(a_t | s_t) - Q_{\theta}(s_t, a_t))] \quad (18)$$

The action a_t is represented by:

$$a_t = f_{\phi}(\varepsilon_t; s_t) = f_{\phi}^{\mu}(s_t) + \varepsilon_t \odot f_{\phi}^{\sigma}(s_t) \quad (19)$$

where ε_t represents noise sampled from a standard normal distribution.

Furthermore, the temperature coefficient α should also realize adaptive updates during the training process by minimizing following $J(\alpha)$:

$$J(\alpha) = E_{a_t \sim \pi_t} [-\alpha \ln(\pi_t(a_t | s_t) - \alpha H_0)] \quad (20)$$

where H_0 is typically set to $-\dim(A)$, representing the dimensionality of the action space. During the interaction between the agent and the environment, Eqs. (16), (18), and (20) are iteratively updated, thereby achieving a gradual convergence of model parameters.

Data availability

Data will be made available on request.

References

- [1] C. Wang, X. Cui, S. Zhao, X. Zhou, Y. Song, Y. Wang, K. Guo, A deep reinforcement learning-based active suspension control algorithm considering deterministic experience tracing for autonomous vehicle, *Appl. Soft Comput.* 153 (2024) 111259.
- [2] J. Fan, X. Zhang, Y. Zou, Hierarchical path planner for unknown space exploration using reinforcement learning-based intelligent frontier selection, *Expert Syst. Appl.* 230 (2023) 120630.
- [3] X. Wang, S. Li, Y. Cao, T. Xin, L. Yang, Dynamic speed trajectory generation and tracking control for autonomous driving of intelligent high-speed trains combining with deep learning and backstepping control methods, *Eng. Appl. Artif. Intell.* 115 (2022) 105230.
- [4] J. Tang, X. Zhang, Y. Zou, Y. Li, G. Du, A high-precision LiDAR-inertial odometry via Kalman filter and factor graph optimization, *IEEE Sens. J.* 23 (11) (2023) 11218–11231.
- [5] S.E. Li, *Reinforcement Learning for Sequential Decision and Optimal Control*, Springer, 2023.
- [6] Z. Zhu, K. Lin, A.K. Jain, J. Zhou, Transfer learning in deep reinforcement learning: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* (2023).
- [7] P. Ladosz, L. Weng, M. Kim, H. Oh, Exploration in deep reinforcement learning: A survey, *Inf. Fusion* 85 (2022) 1–22.
- [8] S. Casper, X. Davies, C. Shi, T.K. Gilbert, J. Scheurer, J. Rando, R. Freedman, T. Korbak, D. Lindner, P. Freire, et al., Open problems and fundamental limitations of reinforcement learning from human feedback, 2023, arXiv preprint arXiv:2307.15217.
- [9] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza, Champion-level drone racing using deep reinforcement learning, *Nature* 620 (7976) (2023) 982–987.
- [10] P. Li, X. Pei, Z. Chen, X. Zhou, J. Xu, Human-like motion planning of autonomous vehicle based on probabilistic trajectory prediction, *Appl. Soft Comput.* 118 (2022) 108499.
- [11] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, B. Zhou, Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (3) (2022) 3461–3475.
- [12] S. Levine, A. Kumar, G. Tucker, J. Fu, *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*, 5, 2020.
- [13] R. Nian, J. Liu, B. Huang, A review on reinforcement learning: Introduction and applications in industrial process control, *Comput. Chem. Eng.* 139 (2020) 106886.

- [14] T.M. Moerland, J. Broekens, A. Plaat, C.M. Jonker, et al., Model-based reinforcement learning: A survey, *Found. Trends® Mach. Learn.* 16 (1) (2023) 1–118.
- [15] L. Kaiser, M. Babaeizadeh, P. Milos, B. Osinski, R.H. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, et al., Model-based reinforcement learning for atari, 2019, arXiv preprint [arXiv:1903.00374](https://arxiv.org/abs/1903.00374).
- [16] J. Wu, Z. Huang, C. Lv, Uncertainty-aware model-based reinforcement learning: Methodology and application in autonomous driving, *IEEE Trans. Intell. Veh.* 8 (1) (2022) 194–203.
- [17] Y. Guan, Y. Ren, H. Ma, S.E. Li, Q. Sun, Y. Dai, B. Cheng, Learn collision-free self-driving skills at urban intersections with model-based reinforcement learning, in: 2021 IEEE International Intelligent Transportation Systems Conference, ITSC, IEEE, 2021, pp. 3462–3469.
- [18] B. Mirchevska, M. Hügle, G. Kalweit, M. Werling, J. Boedecker, Amortized Q-learning with model-based action proposals for autonomous driving on highways, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2021, pp. 1028–1035.
- [19] Q. Huang, Model-based or model-free, a review of approaches in reinforcement learning, in: 2020 International Conference on Computing and Data Science, CDS, IEEE, 2020, pp. 219–221.
- [20] J. Ramírez, W. Yu, A. Perrusquía, Model-free reinforcement learning from expert demonstrations: a survey, *Artif. Intell. Rev.* 55 (4) (2022) 3213–3241.
- [21] D. Bennett, Y. Niv, A.J. Langdon, Value-free reinforcement learning: policy optimization as a minimal model of operant behavior, *Curr. Opin. Behav. Sci.* 41 (2021) 114–121.
- [22] M. Toromanoff, E. Wirbel, F. Moutarde, End-to-end model-free reinforcement learning for urban driving using implicit affordances, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 7153–7162.
- [23] R. Zhang, J. Hou, G. Chen, Z. Li, J. Chen, A. Knoll, Residual policy learning facilitates efficient model-free autonomous racing, *IEEE Robot. Autom. Lett.* 7 (4) (2022) 11625–11632.
- [24] M. Schier, C. Reinders, B. Rosenhahn, Deep reinforcement learning for autonomous driving using high-level heterogeneous graph representations, in: 2023 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2023, pp. 7147–7153.
- [25] J. Li, L. Sun, J. Chen, M. Tomizuka, W. Zhan, A safe hierarchical planning framework for complex driving scenarios based on reinforcement learning, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2021, pp. 2660–2666.
- [26] D. Yarats, A. Zhang, I. Kostrikov, B. Amos, J. Pineau, R. Fergus, Improving sample efficiency in model-free reinforcement learning from images, in: Proceedings of the Aaai Conference on Artificial Intelligence, Vol. 35, 2021, pp. 10674–10681.
- [27] Y. Liu, A. Halev, X. Liu, Policy learning with constraints in model-free reinforcement learning: A survey, in: The 30th International Joint Conference on Artificial Intelligence (IJCAI), 2021.
- [28] Z. Li, F. Liu, W. Yang, S. Peng, J. Zhou, A survey of convolutional neural networks: analysis, applications, and prospects, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (12) (2021) 6999–7019.
- [29] A.G. Schwing, R. Urtasun, Fully connected deep structured networks, 2015, arXiv preprint [arXiv:1503.02351](https://arxiv.org/abs/1503.02351).
- [30] H. Salehinejad, S. Sankar, J. Barfett, E. Colak, S. Valaee, Recent advances in recurrent neural networks, 2017, arXiv preprint [arXiv:1801.01078](https://arxiv.org/abs/1801.01078).
- [31] S. Dong, P. Wang, K. Abbas, A survey on deep learning and its applications, *Comp. Sci. Rev.* 40 (2021) 100379.
- [32] J. Wu, Z. Huang, Z. Hu, C. Lv, Toward human-in-the-loop AI: Enhancing deep reinforcement learning via real-time human guidance for autonomous driving, *Engineering* 21 (2023) 75–91.
- [33] L. Szöke, S. Aradi, T. Bécsi, P. Gáspár, Driving on highway by using reinforcement learning with CNN and LSTM networks, in: 2020 IEEE 24th International Conference on Intelligent Engineering Systems, INES, IEEE, 2020, pp. 121–126.
- [34] L. Li, W. Zhao, C. Xu, C. Wang, Q. Chen, S. Dai, Lane-change intention inference based on RNN for autonomous driving on highways, *IEEE Trans. Veh. Technol.* 70 (6) (2021) 5499–5510.
- [35] A. Vaswani, Attention is all you need, *Adv. Neural Inf. Process. Syst.* (2017).
- [36] Y. Wang, Y. Qiu, P. Cheng, J. Zhang, Transformer-based descriptors with fine-grained region supervisions for visual place recognition, *Knowl.-Based Syst.* 280 (2023) 110993.
- [37] P. Shaw, J. Uszkoreit, A. Vaswani, Self-attention with relative position representations, 2018, arXiv preprint [arXiv:1803.02155](https://arxiv.org/abs/1803.02155).
- [38] Q. Zheng, L. Xu, F. Wang, Y. Xu, C. Lin, G. Zhang, HilbertSCNet: Self-attention networks for small target segmentation of aerial drone images, *Appl. Soft Comput.* 150 (2024) 111035.
- [39] A.B. Adam, L. Lei, S. Chatzinotas, N.U.R. Junejo, Deep convolutional self-attention network for energy-efficient power control in NOMA networks, *IEEE Trans. Veh. Technol.* 71 (5) (2022) 5540–5545.
- [40] K. Liu, Y. Zhao, G. Wang, B. Peng, Self-attention-based multi-agent continuous control method in cooperative environments, *Inform. Sci.* 585 (2022) 454–470.
- [41] A. Wong, Y. Wu, S. Abbasi, S. Nair, Y. Chen, M.J. Shafiee, Fast GraspNet: A fast self-attention neural network architecture for multi-task learning in computer vision tasks for robotic grasping on the edge, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 2293–2297.
- [42] H. Shao, L. Wang, R. Chen, H. Li, Y. Liu, Safety-enhanced autonomous driving using interpretable sensor fusion transformer, in: Conference on Robot Learning, PMLR, 2023, pp. 726–737.
- [43] O. Sharma, N.C. Sahoo, N.B. Puhan, Transformer based composite network for autonomous driving trajectory prediction on multi-lane highways, *Appl. Intell.* (2024) 1–35.
- [44] G. Zhan, Y. Jiang, S.E. Li, Y. Lyu, X. Zhang, Y. Yin, A transformation-aggregation framework for state representation of autonomous driving systems, *IEEE Trans. Intell. Transp. Syst.* (2024).
- [45] L. Ge, X. Zhou, Y. Li, Y. Wang, Deep reinforcement learning navigation via decision transformer in autonomous driving, *Front. Neurobot.* 18 (2024) 1338189.
- [46] J. Lu, L. Han, Q. Wei, X. Wang, X. Dai, F.-Y. Wang, Event-triggered deep reinforcement learning using parallel control: A case study in autonomous driving, *IEEE Trans. Intell. Veh.* 8 (4) (2023) 2821–2831.
- [47] M. Mazouchi, S.P. Nagesh Rao, H. Modares, A risk-averse preview-based q-learning algorithm: Application to highway driving of autonomous vehicles, *IEEE Trans. Control Syst. Technol.* 31 (4) (2023) 1803–1818.
- [48] H.D. Nguyen, K. Han, Safe reinforcement learning-based driving policy design for autonomous vehicles on highways, *Int. J. Control Autom. Syst.* 21 (12) (2023) 4098–4110.
- [49] X. Peng, X. Bu, X. Zhang, M. Dong, K. Ota, Double broad Q-Network for overtaking control of autonomous driving, *IEEE Trans. Veh. Technol.* (2024).
- [50] J. Guo, I. Harmati, Lane-changing system based on deep Q-learning with a request-respond mechanism, *Expert Syst. Appl.* 235 (2024) 121242.
- [51] H. Lai, J. Shen, W. Zhang, Y. Huang, X. Zhang, R. Tang, Y. Yu, Z. Li, On effective scheduling of model-based reinforcement learning, *Adv. Neural Inf. Process. Syst.* 34 (2021) 3694–3705.
- [52] X. Chen, C. Wang, Z. Zhou, K. Ross, Randomized ensembled double q-learning: Learning fast without a model, 2021, arXiv preprint [arXiv:2101.05982](https://arxiv.org/abs/2101.05982).
- [53] L. Smith, I. Kostrikov, S. Levine, Demonstrating a walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning, *Robot.: Sci. Syst. (RSS) Demo 2* (3) (2023) 4.
- [54] K. Li, Y. Wang, J. Zhang, P. Gao, G. Song, Y. Liu, H. Li, Y. Qiao, Uniformer: Unifying convolution and self-attention for visual recognition, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (10) (2023) 12581–12600.
- [55] MetaDrive documentation, 2024, URL <https://metadrive-simulator.readthedocs.io/en/latest/>.
- [56] D. Alexey, An image is worth 16x16 words: Transformers for image recognition at scale, 2020, arXiv preprint [arXiv:2010.11929](https://arxiv.org/abs/2010.11929).
- [57] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631.
- [58] E. Nikishin, M. Schwarzer, P. D’Oro, P.-L. Bacon, A. Courville, The primacy bias in deep reinforcement learning, in: International Conference on Machine Learning, PMLR, 2022, pp. 16828–16847.
- [59] P. D’Oro, M. Schwarzer, E. Nikishin, P.-L. Bacon, M.G. Bellemare, A. Courville, Sample-efficient reinforcement learning by breaking the replay ratio barrier, in: Deep Reinforcement Learning Workshop NeurIPS 2022, 2022.
- [60] M. Janner, J. Fu, M. Zhang, S. Levine, When to trust your model: Model-based policy optimization, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [61] S. Targ, D. Almeida, K. Lyman, Resnet in resnet: Generalizing residual architectures, 2016, arXiv preprint [arXiv:1603.08029](https://arxiv.org/abs/1603.08029).
- [62] J. Xu, X. Sun, Z. Zhang, G. Zhao, J. Lin, Understanding and improving layer normalization, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [63] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International Conference on Machine Learning, PMLR, 2018, pp. 1861–1870.